Comparación experimental de Selection Sort recursivo y Shell Sort iterativo

Experimental comparison of recursive Selection Sort and iterative Shell Sort

Autor: Llano Capia Rodrigo Jhoel

Dependencia: Escuela Profesional de Ingeniería en Informatica y Sistemas, Universidad Nacional Jorge Basadre Grohmann

Afiliación: Tacna, Perú | Correo institucional: rllanoc@unjbg.edu.pe

Resumen

Este trabajo compara el desempeño de Selection Sort recursivo y Shell Sort iterativo mediante experimentos controlados con diferentes tamaños y patrones de entrada. Se evaluaron arreglos de 1 000 y 10 000 elementos para ambos algoritmos, y de 100 000 elementos únicamente para Shell Sort, con entradas aleatorias, ascendentes y descendentes. Cada condición se ejecutó 30 veces y se registraron tiempo de ejecución, comparaciones, intercambios, desplazamientos, profundidad de recursión y corrección del resultado. Los 450 registros obtenidos indicaron que todas las ejecuciones produjeron resultados correctos. En las condiciones comparables, Shell Sort presentó menores tiempos promedio; por ejemplo, para 10 000 elementos aleatorios registró 4,094 ms frente a 356,686 ms de Selection Sort recursivo. Los resultados son coherentes con el costo cuadrático de Selection Sort y muestran que el patrón de entrada influye en el rendimiento observado de Shell Sort. La comparación está limitada por el entorno de ejecución disponible y por no haber ejecutado Selection Sort recursivo con 100 000 elementos.

Palabras clave: algoritmos de ordenamiento; Selection Sort; Shell Sort; análisis experimental; complejidad temporal.

Abstract

This study compares the performance of recursive Selection Sort and iterative Shell Sort through controlled experiments using different input sizes and patterns. Arrays of 1,000 and 10,000 elements were evaluated for both algorithms, while arrays of 100,000 elements were tested only with Shell Sort. Input patterns were random, ascending, and descending. Each condition was executed 30 times, recording execution time, comparisons, swaps, shifts, recursion depth, and output correctness. All 450 recorded runs produced correct results. In comparable conditions, Shell Sort showed lower mean execution times; for example, on 10,000 random elements it took 4.094 ms compared with 356.686 ms for recursive Selection Sort. The results agree with the quadratic cost of Selection Sort and show that input patterns affect the observed performance of Shell Sort. The comparison is limited by the available execution environment and by the absence of a 100,000-element test for recursive Selection Sort.

Keywords: sorting algorithms; Selection Sort; Shell Sort; experimental analysis; time complexity.

1. Introducción

Los algoritmos de ordenamiento organizan elementos de una colección según un criterio y constituyen una herramienta básica para estudiar el diseño y análisis de algoritmos. Aunque dos métodos pueden resolver el mismo problema, su costo en tiempo y operaciones puede variar según su estrategia, el tamaño de la entrada y la distribución inicial de los datos (Cormen et al., 2022).

Selection Sort selecciona repetidamente el menor elemento de la parte no ordenada y lo coloca en su posición final. En su versión tradicional realiza un número cuadrático de comparaciones. Shell Sort, por su parte, ordena elementos separados por intervalos que se reducen progresivamente hasta llegar a un intervalo de uno; su rendimiento depende de la secuencia de intervalos empleada (Cormen et al., 2022; Knuth, 1998).

El objetivo de este estudio es comparar experimentalmente una implementación recursiva de Selection Sort con una implementación iterativa de Shell Sort. Se plantea como hipótesis descriptiva que Shell Sort tendrá menor tiempo de ejecución en las condiciones probadas, especialmente cuando aumente el tamaño de la entrada. No se plantea una hipótesis inferencial ni se pretende generalizar los tiempos a todos los equipos o implementaciones.

2. Método

2.1. Algoritmos y entorno

Los algoritmos se implementaron en C++ utilizando el entorno de desarrollo Dev-C++ y el compilador TDM-GCC 4.9.2 de 64 bits, en configuración Release. Se empleó el estándar C++11 mediante la opción -std=c++11 y la opción de enlazado -static-libgcc. Los experimentos se realizaron en un equipo con procesador Intel Core i9 de 12.ª generación y 32 GB de memoria RAM, bajo el sistema operativo Windows 11 Pro, versión 26H2, compilación 26300.9711. No se seleccionó explícitamente un nivel de optimización en el apartado correspondiente de la configuración del compilador.

2.2. Datos y diseño experimental

Se utilizaron tres patrones de entrada: aleatorio, ascendente y descendente. Para cada combinación de algoritmo, tamaño y patrón se realizaron 30 repeticiones. Ambos algoritmos se probaron con n = 1 000 y n = 10 000. Shell Sort también se probó con n = 100 000; Selection Sort recursivo no se evaluó en ese tamaño debido al elevado costo temporal y a la profundidad de recursión. Los datos aleatorios se generaron con rand() % 100000 y la semilla se calculó mediante la expresión semilla = 12345 + t × 100000 + p × 1000 + rep, donde t es el índice del tamaño, p el índice del patrón y rep el número de repetición, de 1 a 30. Las semillas también se registraron para los patrones ordenados, aunque estos se construyeron directamente y no utilizaron números aleatorios. Shell Sort empleó una secuencia de intervalos que comienza en n/2

y se divide entre dos hasta llegar a uno. Para reproducir el experimento se debe conservar el código fuente y utilizar el mismo entorno de compilación y ejecución.

El conjunto contiene 450 registros: 30 repeticiones para cada una de 15 combinaciones de algoritmo, tamaño y patrón. La variable correcto indica si el arreglo quedó ordenado correctamente; todos los registros tienen el valor “SI”.

2.3. Métricas

Se registraron tiempo de ejecución en milisegundos, comparaciones, intercambios, desplazamientos, profundidad de recursión y corrección. Para el tiempo se calcularon media, mediana, desviación estándar, mínimo y máximo. Las operaciones se resumen mediante el promedio de las repeticiones de cada condición. El contador de desplazamientos de Shell Sort registra movimientos de elementos y la colocación del elemento temporal según la implementación; por eso, su contador de intercambios permanece en cero. La profundidad informada es la medida que implementa el programa y no una medición directa de consumo de memoria.

3. Resultados

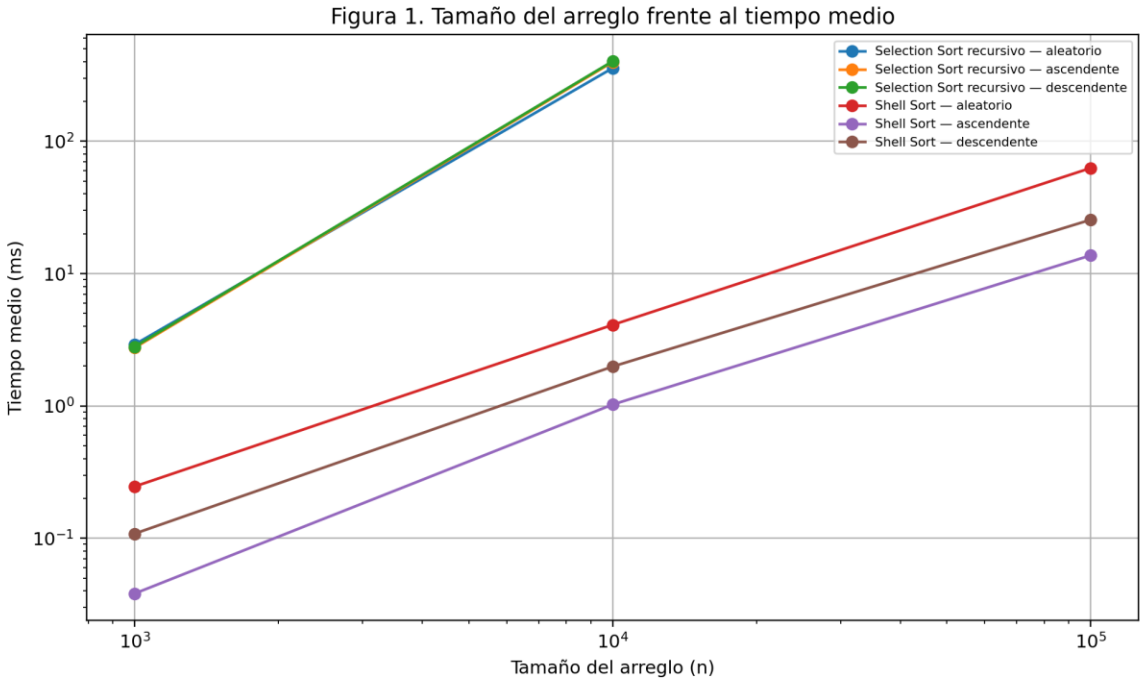
Los 450 registros fueron marcados como correctos. En las seis condiciones comparables con n = 1 000 y las seis con n = 10 000, Shell Sort obtuvo menor tiempo medio que Selection Sort recursivo. La Tabla 1 presenta las estadísticas descriptivas de tiempo.

Tabla 1. Estadísticas descriptivas del tiempo de ejecución (ms).

Algoritmo	n	Patrón	Media	Mediana	Desv. est.	Mín.	Máx.
Shell Sort	1 000	aleatorio	0.246	0.243	0.016	0.231	0.321
Selection Sort recursivo	1 000	aleatorio	2.910	2.819	0.428	2.785	5.168
Shell Sort	1 000	ascendente	0.038	0.038	0.002	0.037	0.047
Selection Sort recursivo	1 000	ascendente	2.751	2.743	0.047	2.677	2.891
Shell Sort	1 000	descendente	0.108	0.108	0.001	0.107	0.112
Selection Sort recursivo	1 000	descendente	2.800	2.738	0.236	2.678	3.966
Shell Sort	10 000	aleatorio	4.094	4.037	0.586	3.692	7.034
Selection Sort recursivo	10 000	aleatorio	356.686	391.426	61.322	265.118	460.977
Shell Sort	10 000	ascendente	1.024	1.019	0.032	0.984	1.109
Selection Sort recursivo	10 000	ascendente	395.986	393.498	7.412	390.013	413.969
Shell Sort	10 000	descendente	1.983	1.967	0.082	1.911	2.376
Selection Sort recursivo	10 000	descendente	402.019	400.563	5.407	396.085	417.624
Shell Sort	100 000	aleatorio	62.693	62.275	3.071	58.399	73.451
Shell Sort	100 000	ascendente	13.717	13.106	1.649	12.620	19.998
Shell Sort	100 000	descendente	25.491	25.221	0.912	24.462	28.600

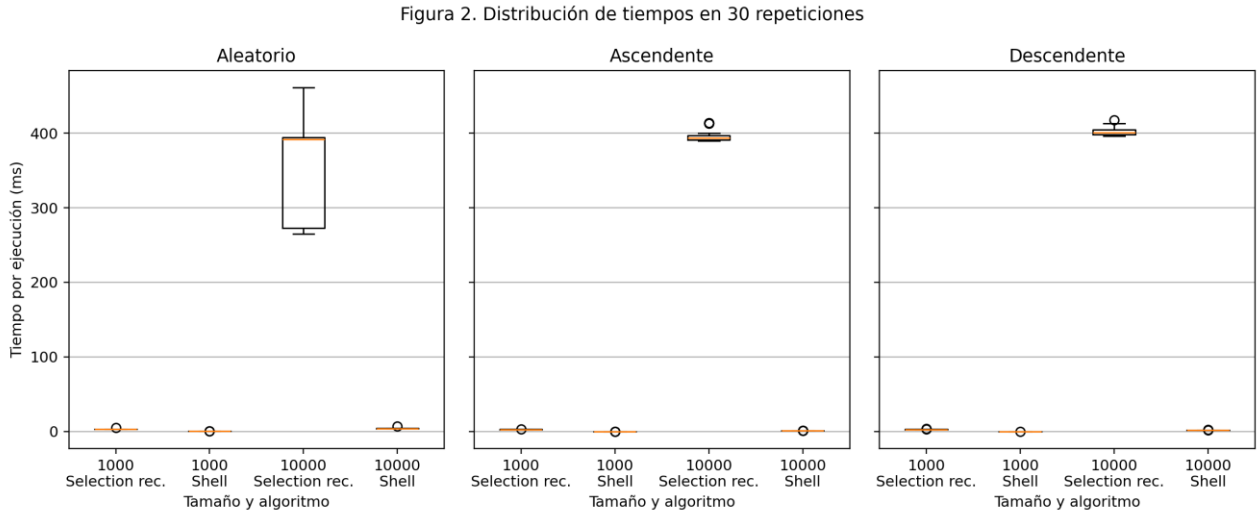
Nota. Cada estadística se calculó sobre 30 repeticiones por condición. Fuente: elaboración propia a partir de resumen.csv.

Figura 1. Tiempo medio de ejecución según tamaño y patrón de entrada.



La Figura 1 muestra que el tiempo aumenta con el tamaño de entrada. Para n = 10 000 y patrón aleatorio, Selection Sort recursivo alcanzó una media de 356,686 ms, frente a 4,094 ms de Shell Sort. En n = 100 000 solo se dispone de resultados de Shell Sort, por lo que ese tramo no permite una comparación directa entre los dos algoritmos.

Figura 2. Distribución de los tiempos de ejecución por algoritmo y tamaño.



La Figura 2 presenta la dispersión de los tiempos medidos en las condiciones representadas. La variabilidad no fue idéntica entre condiciones; por ejemplo, Selection Sort recursivo con 10 000

elementos aleatorios tuvo una desviación estándar de 61,323 ms, mientras que Shell Sort en esa condición presentó 0,586 ms.

Tabla 2. Promedio de operaciones registradas y profundidad de recursión.

Algoritmo	n	Patrón	Comparaciones	Intercambios	Desplazamientos	Prof. recursión
Selection Sort recursivo	1 000	aleatorio	499 500	992.2	0.0	999
Selection Sort recursivo	1 000	ascendente	499 500	0.0	0.0	999
Selection Sort recursivo	1 000	descendente	499 500	500.0	0.0	999
Selection Sort recursivo	10 000	aleatorio	49 995 000	9 990.6	0.0	9 999
Selection Sort recursivo	10 000	ascendente	49 995 000	0.0	0.0	9 999
Selection Sort recursivo	10 000	descendente	49 995 000	5 000.0	0.0	9 999
Shell Sort	1 000	aleatorio	15 231	0.0	11 745.0	0
Shell Sort	1 000	ascendente	8 006	0.0	0.0	0
Shell Sort	1 000	descendente	11 716	0.0	9 072.0	0
Shell Sort	10 000	aleatorio	264 523	0.0	207 585.2	0
Shell Sort	10 000	ascendente	120 005	0.0	0.0	0
Shell Sort	10 000	descendente	172 578	0.0	124 592.0	0
Shell Sort	100 000	aleatorio	4 233 328	0.0	3 546 962.4	0
Shell Sort	100 000	ascendente	1 500 006	0.0	0.0	0
Shell Sort	100 000	descendente	2 244 585	0.0	1 655 808.0	0

Nota. Los valores representan promedios de 30 ejecuciones y se redondean para facilitar la lectura. Fuente: elaboración propia a partir de resultados.csv.

4. Discusión

Los resultados respaldan la hipótesis descriptiva planteada para las condiciones observadas: Shell Sort fue más rápido que Selection Sort recursivo en todos los pares comparables. La diferencia es especialmente amplia para n = 10 000, donde Selection Sort registró 49 995 000 comparaciones en cada patrón. Esta cifra coincide con la cantidad de comparaciones característica de la implementación clásica de Selection Sort, $n(n-1)/2$ (Cormen et al., 2022).

Shell Sort presentó cantidades de comparaciones y tiempos diferentes según el patrón de entrada. En los datos medidos, la entrada ascendente produjo el menor tiempo medio para cada tamaño de

Shell Sort. Esto describe esta implementación y la secuencia de intervalos utilizada, y no implica que todas las variantes de Shell Sort tengan el mismo comportamiento.

En cuanto a estabilidad, las implementaciones tradicionales de Selection Sort y Shell Sort se consideran inestables: elementos con claves iguales pueden cambiar su orden relativo. El experimento no incluyó una prueba aislada que rastreara la identidad y el orden relativo de elementos duplicados, por lo que la estabilidad se describe como propiedad del algoritmo/implementación convencional y no como un resultado medido directamente en esta ejecución.

Existen limitaciones importantes. Primero, no se ejecutó Selection Sort recursivo con n = 100 000, de modo que no hay comparación directa para ese tamaño. Segundo, los tiempos pueden depender de la carga del equipo, las opciones efectivas de compilación y la implementación exacta. Los experimentos se realizaron con Dev-C++, TDM-GCC 4.9.2 de 64 bits, C++11 y Windows 11 Pro, versión 26H2, compilación 26300.9711; no se seleccionó explícitamente un nivel de optimización en el apartado correspondiente de la configuración del compilador. Finalmente, la profundidad de recursión reportada no sustituye una medición del uso real de pila o memoria.

5. Conclusiones

1. En todas las condiciones comparables, Shell Sort presentó un tiempo promedio inferior al de Selection Sort recursivo.
2. Para n = 10 000 con datos aleatorios, Shell Sort tardó en promedio 4,094 ms y Selection Sort recursivo 356,686 ms.
3. Selection Sort recursivo registró 499 500 comparaciones para n = 1 000 y 49 995 000 para n = 10 000, consistente con un crecimiento cuadrático.
4. Shell Sort mostró tiempos y cantidades de operaciones distintos según el patrón de entrada; la entrada ascendente fue la más rápida en las pruebas realizadas.
5. Todas las ejecuciones registradas ordenaron correctamente los datos, pero el estudio debe completarse con los datos exactos del entorno de compilación y no permite comparar ambos algoritmos para n = 100 000.

Agradecimientos

Se agradece la orientación docente brindada en el curso de Programación Avanzada para el desarrollo del análisis experimental.

Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press.